Bywaf Usage Guide

Overview

Bywaf is a highly-auditable Python 3 commandlet framework for authorized web application and network testing workflows. It presents a Metasploit-like interactive shell, loads commandlets from plugins, and connects commandlets through a SQLite-backed event bus.

The core idea is simple: one commandlet discovers something and publishes it as an event; another commandlet consumes that event and publishes the next result. For example, `hostscanner` can publish live hosts, `portscanner` can consume those hosts and publish open ports, and HTTP commandlets can consume open ports and probe web services.

Use Bywaf only on systems and networks where you have explicit authorization.

Evolving framework design notes are tracked in DESIGN.md. Core architectural references:

- TERMINOLOGY.md defines jobs, pipelines, runs, events, topics, commandlets, plugins, capabilities, local IDs, and serials.
- RUNTIME_MODEL.md explains runtime entities, lifecycle, foreground/background execution, control signals, and variable snapshots.
- EVENT_MODEL.md explains event rows, topics, replay, framework requests, artifacts, notes, and provenance.
- CAPABILITY_MODEL.md explains capability auditing, policy direction, and plugin integration types.
- SYSTEM_BLOCK_DIAGRAM.pdf shows live runtime flow and durable data flow through the system.
- SYSTEM_DATAFLOW_DIAGRAM.pdf focuses on command input, event, artifact, audit, request, and report data movement.

Installation

During development, run Bywaf from the repository root:

```
cd bywaf
python3 -m bywaf --help
python3 -m bywaf repl
```

For an editable local install:

```
python3 -m pip install -e .
bywaf --help
bywaf repl
```

For a local Debian package build, install the Debian build dependencies and run `dpkg-buildpackage` from the repository root:

```
sudo apt install debhelper dh-python pybuild-plugin-pyproject python3-all python3-setuptools
dpkg-buildpackage -us -uc -b
sudo apt install ../bywaf_0.9.0-1_all.deb
bywaf --help
```

The project metadata defines a console script named `bywaf`, so packaged installations can expose Bywaf as a normal command instead of requiring `python3 -m bywaf`.

Bywaf can also be embedded as a Python library. A local GUI or web service should use the public session facade instead of scraping REPL output:

```
from pathlib import Path
from bywaf import BywafSession

session = BywafSession.open(Path(".bywaf/bywaf.sqlite3"))
session.run("hostscanner 127.0.0.1")
hosts = session.events(topic="host.found")
jobs = session.jobs()
```

The host and port scanner commandlets use `nmap` through a Python adapter. A local `nmap` binary is required for real scans. The adapter prefers `nmaplib`, then `python-nmap`, then `nmapthon`, then `libnmap`.

Dependency summary:

```
nmap                        required for hostscanner and portscanner
prompt_toolkit              required for rich interactive REPL completion
nmaplib/python-nmap/etc.    Python nmap adapter; Bywaf tries supported adapters
sqlcipher3-binary           optional Python SQLCipher driver for encrypted DBs
sqlcipher                   optional system SQLCipher tooling/library
scapy                       optional helper library for future packet plugins
```

Bywaf plugins are intended to wrap useful external tools and normalize their results into the central event database. That removes manual handoffs such as copying hosts to notes or intermediate files: one plugin can discover hosts, another can consume those host events, and later plugins can continue the workflow from the same stored data.

Execution-time plugin variables are scoped by commandlet. A plugin uses `context.vars.get("name")` for its own variables and cannot enumerate another plugin's variables through that API. Explicit global variables use `context.vars.get_global("name")`. When a commandlet run starts, Bywaf snapshots the effective commandlet and global variables into SQLite under that `command_run_id`; `event run=<id>` displays the captured variables so runs remain auditable and reproducible even when session variables change later. Runtime entities have two identities: local IDs for interactive typing (`job=12`, `run=1`, `pipeline=2`) and durable serials for audit/provenance. Local IDs are stable inside the current database and are never reused there, but they are not portable across replay/import into another database. Use `event`

`serial=<serial>` when you want to inspect by the durable identifier. Explicit `load plugin=...` and `load script=...` operations also receive resource serials, so the load itself and the script commands it executed can be reviewed later.

Plugins that need interpreter-owned actions use request events instead of direct method calls. For example, a plugin can publish `shell.prompt.requested`; the foreground REPL validates the request and records either `shell.prompt.updated` or `framework.request.denied` for auditability. Plugins also declare intended capabilities on `CommandSpec`; Bywaf records audit-only `plugin.capability.used` and `plugin.capability.missing` events so operators can compare intended behavior with actual behavior. Plugin event-bus access should go through `context.events`, which audits `db.read:<topic>` and `db.write:<topic>` capability usage. Raw `context.db` access is retained for privileged/internal framework commandlets during the transition and audits `db.raw`.

Encrypted databases require SQLCipher support. On Debian or Ubuntu, install the SQLCipher library and use the optional Python extra:

```
sudo apt install sqlcipher libsqlcipher-dev
python3 -m pip install -e '.[sqlcipher]'
```

Starting Bywaf

Start the interactive shell:

```
bywaf
```

or, from a source checkout:

```
python3 -m bywaf
```

Create or open the default database with SQLCipher encryption:

```
bywaf --encrypt
bywaf --database client.sqlite3 --encrypt
```

Run one command non-interactively:

```
bywaf run ls
bywaf run cat README.md
bywaf run 'hostscanner 127.0.0.1 | portscanner'
```

Simple `run` commands do not need quotes. Use quotes when the command contains shell metacharacters such as `|`, `&`, `>`, or spaces that must be preserved inside a single argument.

REPL Basics

The REPL prompt is:

```
bywaf>
```

Commandlets can be run directly:

```
bywaf> ls
bywaf> cat README.md
bywaf> hostscanner 127.0.0.1
```

Built-in commands manage the shell and the event database:

```
help
help <command>
plugins
cmds
vars
history
info
job <list|show|cancel|end|kill>
pipeline <list|show|cancel|end|kill>
signal <job=id|run=id|serial=id> <action> [--soft|--hard] [key=value ...]
cancel <job=id|pipeline=id|run=id>
end [--soft|--hard] <job=id|pipeline=id|run=id>
kill [--soft|--hard] <job=id|pipeline=id|run=id>
jobs
runs
events [tail|--tail] [last=N]
topics
db <status|path|checkpoint|vacuum|new|encrypt|decrypt|rekey>
event <topic|job=id|run=id|pipeline=id|serial=id>
load <resource>
save <resource>
exit
```

`help <command>` shows the same help as `<command> --help` for commandlets.

Commandlets

A commandlet is an executable unit exposed by a plugin. Each commandlet declares its name, description, arguments, consumed topics, and emitted topics.

Examples:

```
bywaf> help hostscanner
bywaf> hostscanner --help
bywaf> portscanner --help
```

Many scanning commandlets support `-s` or `--silent` to suppress console alerts. Commandlets request alerts through the framework using the database; the framework validates the request, stores a structured `console.alert` event, and prints it unless silent mode is active:

```
hostscanner <hostscanner-...>: discovered host 127.0.0.1
portscanner <portscanner-...>: discovered port 127.0.0.1:80/tcp
```

Commandlets can also declare tab-completion behavior for their arguments and options. For example, `ls [path]`, `cat <path>`, and `less <path>` get filename completion because those commandlets declare path/file completion in their plugin specs. Other completion specs include `topic`, `run`, `pipeline`, `job`, and `plugin`, so plugin authors can make hand-typed commands much easier to complete correctly. Runtime entity completions include prompt-toolkit metadata when available, such as serial, status/source, event counts, and the current number of attached artifacts.

Interactive shells use `prompt_toolkit` when a real terminal is available. `Ctrl-Space` enters completion-selection mode by opening the menu and selecting the first candidate. Then arrow keys move through candidates, `Enter` selects the highlighted completion, and `Esc` returns to the command line. A bottom toolbar shows that hint while a completion menu is open. The selection-mode key is configurable with `vars completion.select-key=<key>` using prompt-toolkit key names, because some desktop environments or terminal stacks reserve `Ctrl-Space`. `vars completion.wasd-selection=true` enables optional WASD-style menu navigation (`w`/`a` move backward, `s`/`d` move forward, following prompt-toolkit's flat completion order), but it is off by default so ordinary typing is not intercepted. Minimal non-interactive environments fall back to readline-style completion.

Plugin authors should use `context.output()`, `context.table()`, `context.alert()`, `context.progress()`, `context.page_file()`, and `context.process` instead of direct `print()` calls, direct terminal control, or direct subprocess calls. These helpers keep terminal output, progress, and external tool execution auditable and make the same commandlets usable from a future GUI or web frontend.

Progress is separate from findings. A finding is durable evidence such as `host.found` or `port.open`; progress is operational state such as "42% through the TCP scan." Commandlets report progress through structured events:

```
context.progress_started(phase="tcp_scan", total=1000, unit="ports")
context.progress(phase="tcp_scan", current=420, total=1000, unit="ports")
context.progress_completed(phase="tcp_scan", current=1000, total=1000, unit="ports")
```

Bywaf enforces progress throttling in the framework. Configure it with global session variables:

```
bywaf> vars global.progress.min-interval-ms=250
bywaf> vars global.progress.min-percent-delta=1
```

Audit logs are stored as SQLite events. Use `audit show ...` to inspect them and `audit export file=audit.jsonl`, `audit export file=audit.pdf`, or `audit export file=audit.sqlite3` to hand off a copy.

```
bywaf> audit show topic=console.alert since=20260517 until=20260518
bywaf> audit export file=audit.pdf since=run=<command-run-id>
bywaf> audit export --encrypt file=audit.sqlite3
bywaf> audit export --encrypt file=audit.pdf
```

Unqualified `since=` and `until=` audit bounds default to `time:`. Encrypted SQLite audit exports use SQLCipher. Encrypted PDF export uses `pikepdf` when available, otherwise the external `qpdf` command.

Plugins

A plugin provider groups related commandlets. The `plugins` command lists loaded providers:

```
bywaf> plugins
discovery
http
network
os
runtime
storage
```

The `cmds` command lists commandlets grouped by provider:

```
bywaf> cmds
discovery
  hostscanner
http
  http_headers
  http_probe
network
  portscanner
os
  cat
  less
  ls
runtime
  job
storage
  db
```

Bundled plugins are listed in `bywaf/plugins/plugins.json`. Adding a plugin file is not enough to load it by default; add its dotted path to that config.

Load an additional plugin by name from `.bywaf/plugins`:

```
bywaf> load plugin=myplugin
```

Load a plugin from an explicit filesystem path:

```
bywaf> load plugin=./plugins/myplugin
bywaf> load plugin=~/bywaf-plugins/myplugin
```

Pipelines

Pipelines connect commandlets with `|`. Prefix the expression with `name:` to name the pipeline without consuming commandlet arguments:

```
bywaf> hostscanner 127.0.0.1 | portscanner
bywaf> client subnet scan: hostscanner 127.0.0.1 | portscanner
```

The runner executes each stage in order. Events emitted by one stage are passed to the next stage as input. Events are also stored in SQLite with a pipeline ID and command run ID.

This model allows downstream commandlets to consume only the output relevant to the current pipeline, rather than every historical event in the database.

Attach a new background commandlet to an existing pipeline:

```
bywaf> pipeline attach <pipeline-id> portscanner run=<producer-run-id> since=beginning
bywaf> pipeline attach <pipeline-id> http_probe since=now
```

The attach selectors are orthogonal:

- `<pipeline-id>` chooses the pipeline the new command run joins.
- `run=<producer-run-id>` optionally narrows input to one upstream producer run.
- `since=beginning` replays matching historical events, then listens for new events.
- `since=now` ignores historical events and starts from the current event high-water mark.

If `run=` is omitted, the attached commandlet reads matching events from the whole pipeline.

Runtime Names

Name the current command run with a stage-local `name=` selector:

```
bywaf> hostscanner 127.0.0.1 name=localhost sweep
```

Name or inspect runtime entities after they exist:

```
bywaf> name run=<command-run-id> localhost sweep
bywaf> name pipeline=<pipeline-id> client subnet scan
bywaf> name job=<job-id> background listener
bywaf> name run=<command-run-id>
```

The explicit keyed form is `text=`, for example `name run=<id> text=localhost sweep`.

Assigned names appear in `runs`, `pipelines`, and `jobs` listings.

Framework Notes

Any commandlet stage can include a framework-level `note=` selector. The runner strips it before the commandlet receives arguments and records an audited `note.attached` event with the job, pipeline, and command-run IDs.

```
bywaf> hostscanner 10.0.0.0/24 note=client-approved internal subnet
```

If `note=` is the last selector in a stage, it consumes the rest of that stage without requiring quotes:

```
bywaf> hostscanner targets note=scope approved | portscanner note=top ports
```

Review attached notes with the `note` commandlet. Output and file exports use timestamp-first lines:

```
bywaf> note run=<command-run-id>
bywaf> note pipeline=<pipeline-id>
bywaf> note job=<job-id> file=notes.txt
bywaf> note add run=<command-run-id> text=follow-up note
```

Notes are append-only. Adding another note creates another timestamped `note.attached` event instead of replacing earlier notes.

Encrypted Artifacts

Artifacts are evidence files attached to a run, pipeline, or job. Artifact bodies are stored in a separate encrypted SQLCipher database next to the main database, using the main encrypted database passphrase for the session. The main database stores timestamped provenance events such as `artifact.attached` and `artifact.exported`; it does not store artifact bodies. Bywaf derives the artifact DB path from the active main DB path so the two files remain an integrity pair; arbitrary artifact DB switching is intentionally not exposed by default.

Start Bywaf with an encrypted database before attaching artifacts:

```
bywaf --encrypt
```

Attach one or more files:

```
bywaf> artifact attach run=<command-run-id> file=snapshot.html name='Landing page'
bywaf> artifact attach serial=<run-or-pipeline-or-job-serial> file=snapshot.html
bywaf> artifact attach run=<command-run-id> file=snapshot.html file=headers.txt
bywaf> artifact attach pipeline=<pipeline-id> file=report.json note=initial report
```

List, search, save, and verify artifacts:

```
bywaf> artifact list run=<command-run-id>
bywaf> search run=<command-run-id> name=landing
bywaf> search run=<command-run-id> content=csrf
bywaf> artifact search run=<command-run-id> --regexp note='landing|headers'
bywaf> artifact replace artifact=1 file=snapshot-v2.html
bywaf> artifact remove artifact=1
bywaf> artifact save artifact=1 file=snapshot.html
bywaf> artifact save serial=<artifact-serial> file=snapshot.html
bywaf> artifact save run=<command-run-id> dir=artifacts/
bywaf> artifact verify pipeline=<pipeline-id>
```

Use `name=` for human-readable artifact labels; if omitted, the source filename is used. The `search` commandlet, and its `artifact search` alias, search artifact metadata quickly. `name=`, `note=`, and `content=` narrow the search to artifact names, notes, or decoded text contents. Add `--regexp` to treat those field values as Python regular expressions. `since=` and `until=` restrict matches by artifact creation time. Use `file=` when saving exactly one artifact. Use `dir=` when saving a set. If `file=` matches multiple artifacts, Bywaf reports that clearly and asks you to use `dir=` instead. For `artifact attach`, `serial=` may refer to a run, pipeline, or job serial. Artifact serials identify existing artifact rows for listing, searching, saving, and verifying; artifacts are not attached to other artifacts.

At-File Arguments

Any commandlet argument can use framework-level at-file expansion. Expansion happens before the commandlet receives its argument list and is audited as a framework argument expansion. When the active database is encrypted and artifact storage is available, the expanded input file is also attached as an encrypted provenance artifact.

```
bywaf> hostscanner @lines:targets.txt
bywaf> hostscanner 192.168.1.1-255 except=@lines:do-not-scan.txt
bywaf> http_probe @target.txt
bywaf> echo @@literal-at-sign
```

Supported forms:

- `@file`: read the file as one text argument
- `@raw:file`: read the file as one text argument
- `@lines:file`: expand each non-empty line into a separate argument

- `@@value`: pass `@value` literally

Scanner commandlets support `except=` for exclusion lists. Values may be comma separated or file-backed through `@lines:`.

Tab completion preserves these prefixes while completing filesystem paths.

Variable Expansion

Bywaf expands `$variables` before commandlets receive their argument list. Unquoted and double-quoted variables expand; single-quoted variables are passed literally.

```
bywaf> use hostscanner
bywaf> vars targets=192.168.1.1 192.168.1.2
bywaf> hostscanner $targets
bywaf> hostscanner "$targets"
bywaf> hostscanner '$targets'
```

Resolution checks the exact variable name first, then the active commandlet scope, then `global.`. For example, `$targets` in `hostscanner` checks `targets`, `hostscanner.targets`, and `global.targets`. Variable expansion is audited as `framework.variable.expanded`.

Plans And Policy

Commandlets can expose a framework-owned plan hook. Use `--test` to show the intended action and exit without running, or `--yes` to approve a required plan non-interactively:

```
bywaf> hostscanner 192.168.1.0/24 --test
bywaf> hostscanner 192.168.1.0/24 --yes
```

Plans are audited as `plan.requested`, evaluated as `policy.evaluated`, and approved or denied as `plan.approved` / `plan.denied` with `approved_by=<os user>`. Suggested repairs, such as pruning out-of-scope targets for one run, are audited as `plan.repair.applied` or `plan.repair.denied`.

Initial network policy variables:

```
bywaf> vars global.policy.network.allow=192.168.1.0/24
bywaf> vars global.policy.network.deny=169.254.169.254/32,192.168.1.50
bywaf> vars global.plan.required=true
```

The policy layer applies to the run being launched. Repairs do not mutate source files, saved variables, or command history.

Command Continuation And Sequences

Use a trailing backslash to continue a command across physical lines in the interactive shell or in scripts:

```
bywaf> hostscanner \
... 192.168.0.0/24
```

Separate multiple commands with semicolons when they should run sequentially:

```
bywaf> vars target=127.0.0.1; vars; topics
```

Semicolons inside quotes are preserved as part of the argument text.

Background Execution

Append `&` to background a commandlet or pipeline:

```
bywaf> hostscanner 192.168.0.1-255 &
```

Normal commandlet execution is job-audited through the database. Foreground commandlets run in-process but still record `job.requested`, `job.claimed`, `job.started`, and `job.finished` or `job.failed`. Background jobs use the same job lifecycle, but a worker process claims and runs the queued job. Foreground management commands such as `db ...` and `job ...` run directly.

Stage-level backgrounding works inside pipelines:

```
bywaf> hostscanner 192.168.0.1-255 & | portscanner &
```

In that example, `portscanner` listens for `host.found` rows created by the immediately upstream `hostscanner` run in the same pipeline. It does not consume unrelated `host.found` rows from older scans.

A pipeline groups one or more runs in the same command expression or attached workflow. A run is one commandlet invocation inside that pipeline, such as the specific `hostscanner` stage or `portscanner` stage. A job is the supervised foreground/background execution lifecycle that runs one or more of those commandlet invocations. Operationally, jobs are chained together into pipelines by the runs they supervise; one job may contribute the whole chain, or multiple jobs may contribute runs when commandlets are attached later. See `TERMINOLOGY.md` for the canonical definitions of jobs, pipelines, runs, local IDs, serials, events, and topics.

Show the currently active runtime entities:

```
bywaf> info
```

List active jobs, or all jobs with an explicit active marker:

```
bywaf> job list
bywaf> job list --all
bywaf> jobs --all
```

Show one job:

```
bywaf> job show <id>
```

Soft-cancel a job so commandlets that check cancellation can exit cleanly:

```
bywaf> job cancel <id>
```

End a job. By default this is cooperative, like `cancel`; add `--hard` to force-stop the process:

```
bywaf> job end <id>
bywaf> job end --hard <id>
bywaf> job kill --hard <id>
```

Pipelines can be inspected and controlled the same way:

```
bywaf> pipeline list
bywaf> pipeline list --all
bywaf> pipeline show <id>
bywaf> pipeline cancel <id>
bywaf> pipeline end <id>
bywaf> pipeline kill --hard <id>
```

`job list`, `runs`, and `pipeline list` show active runtime state by default. Use `--all` to include historical entries. These commands render table views with local ID, durable serial, lifecycle state, names, timestamps, and an `ARTIFACTS` column counting artifacts attached so far. Set `vars global.listing.active-format=long` to include the state timestamp in the state column; set it to `short` for compact lifecycle labels.

For live runtime control, `signal` is the canonical command for a concrete receiver: a job, a command run, or a `serial=` that resolves to one of those. A pipeline is a grouping scope, not executing code, so it does not receive plugin-domain signals directly. Use pipeline-aware commands such as `pause pipeline=...` or `end --hard pipeline=...` when you want the framework to fan out control over jobs associated with a pipeline. Framework-native signals such as `pause`, `resume`, `stop`, `end`, and `kill` apply the existing framework controls; plugin-domain signals such as `prune`, `mute`, `unmute`, and `verbosity` are delivered for commandlets to apply or ignore.

```
bywaf> signal run=<command-run-id> prune targets=192.168.1.0/24
bywaf> signal run=<command-run-id> mute
bywaf> signal serial=<run-or-job-serial> verbosity level=debug
bywaf> signal run=<command-run-id> verbosity level=debug
bywaf> signal run=<command-run-id> pause --hard
```

`cancel`, `end`, `kill`, `pause`, `resume`, and `stop` are convenience aliases over the same signal/control path. `end` and `kill` are synonyms; both default to cooperative `--soft`, and `--hard` force-terminates the affected process:

```
bywaf> cancel job=<id>
bywaf> cancel pipeline=<id>
bywaf> end job=<id>
bywaf> kill --hard pipeline=<id>
bywaf> pause job=<id>
bywaf> pause --hard job=<id>
bywaf> pause run=<command-run-id>
bywaf> resume --listonly pipeline=<id>
bywaf> resume --listonly run=<command-run-id>
bywaf> stop --hard job=<id>
```

`jobs` remains as a convenience alias for `job list`, and `pipelines` remains as a convenience alias for `pipeline list`.

Database and Event Model

Bywaf stores events in SQLite. The default database is:

`.bywaf/bywaf.sqlite3`

SQLite is used in WAL mode. Inserts are committed immediately; there is no separate document-style save step. On shutdown, Bywaf checkpoints the WAL using `PRAGMA wal_checkpoint(TRUNCATE)` so WAL contents are folded back into the main database file.

List known event topics:

```
bywaf> topics
```

Show the last 25 events, or choose an explicit tail size:

```
bywaf> events
bywaf> events tail
bywaf> events tail last=50
```

Show recent events for a topic:

```
bywaf> event host.found
bywaf> event port.open
```

List command runs:

```
bywaf> runs
```

Show events by command run or pipeline:

```
bywaf> event run=1
bywaf> event pipeline=1
bywaf> event serial=<durable-serial>
```

Save a database snapshot:

```
bywaf> save db=snapshot.sqlite3
```

Save an encrypted snapshot:

```
bywaf> save --encrypt db=snapshot.sqlite3
```

Inspect or maintain the active database:

```
bywaf> db status
bywaf> db path
bywaf> db checkpoint
bywaf> db vacuum
```

Create a fresh database and switch the active session to it:

```
bywaf> db new
bywaf> db new --file=client.sqlite3
bywaf> db new --encrypt --file=client.sqlite3
bywaf> db new --force --file=client.sqlite3
```

Without `--file`, `db new` creates a timestamped database under `.bywaf/db/`. With `--file`, it refuses to overwrite an existing file. Add `--force` to move the existing database and SQLite sidecar files to timestamped `.bak-*` names before creating the new DB. `--encrypt` forces SQLCipher encryption and prompts twice for a passphrase.

The session variable `db.encryption=sqlcipher` makes `db new` encrypted by default:

```
bywaf> vars db.encryption=sqlcipher
bywaf> db new
```

Convert the active database in place:

```
bywaf> db encrypt
bywaf> db rekey
bywaf> db decrypt
```

`db encrypt` converts the active plaintext database to SQLCipher and prompts twice for a new passphrase. `db rekey` changes the passphrase for an encrypted database. `db decrypt` exports the active encrypted database back to plaintext SQLite after an explicit `YES` confirmation.

Switch to another database:

```
bywaf> load db=snapshot.sqlite3
```

If the database is encrypted, Bywaf prompts for its passphrase when loading it. Passphrases are kept in process memory only and are not written to config or history files.

Resource Files

Bywaf keeps default state in:

```
.bywaf/
```

Important files:

```
.bywaf/bywaf.sqlite3
.bywaf/config.json
.bywaf/history.bywaf
.bywaf/plugins/
```

Resource resolution rules are consistent:

```
plugin=<name>   -> .bywaf/plugins/<name>
script=<name>   -> ./<name>
db=<name>       -> ./<name>
config=<name>   -> ./<name>
history=<name>  -> ./<name>
```

Explicit paths are used as filesystem paths for every resource type:

```
./name
../name
~/name
/absolute/name
```

Examples:

```
bywaf> load script=scan.bywaf
bywaf> load script=./scripts/scan.bywaf
bywaf> save config=session.json
bywaf> load config=session.json
bywaf> save history=session-history.bywaf
bywaf> load history=session-history.bywaf
```

Variables

List variables:

```
bywaf> vars
```

Set a variable:

```
bywaf> vars name=value
```

Show one variable:

```
bywaf> vars name
```

Common examples:

```
bywaf> vars http_probe.cookie-file=/tmp/cookies.txt
bywaf> vars history.timestamp-format=%Y-%m-%d %H:%M:%S %Z
bywaf> vars hostscanner.targets=192.168.1.1-255
bywaf> hostscanner
```

Use a commandlet context to make short variable assignments target that commandlet:

```
bywaf> use hostscanner
bywaf> vars targets=192.168.1.1-255
bywaf> use global
```

For commandlets that opt into variable defaults, explicit command-line arguments take precedence over commandlet variables, and commandlet variables take precedence over built-in defaults. For example, `hostscanner 127.0.0.1` ignores `hostscanner.targets`, while `hostscanner` falls back to it.

Save variables:

```
bywaf> save config=config.json
```

Load variables:

```
bywaf> load config=config.json
```

Config files are JSON objects containing session variables.

History

Every non-empty REPL command is appended to:

```
.bywaf/history.bywaf
```

History lines are stored as commands followed by a timestamp comment:

```
hostscanner 127.0.0.1  # 2026-05-12 10:15:30 EDT
```

The `history` command shows only commands from the current REPL invocation, with timestamps displayed first for easier scanning:

```
bywaf> history
2026-05-17 10:00:00 EDT  plugins
```

Limit session history by timestamp with `since=` and `until=`. Unqualified values default to `time:` and use `yyyymmdd[HH[MM[SS]]]`:

```
bywaf> history since=20260517 until=20260518
bywaf> history since=time:202605171000 until=time:202605171059
```

The persistent history file can be viewed with the OS commandlets:

```
bywaf> cat .bywaf/history.bywaf
bywaf> less .bywaf/history.bywaf
```

The persistent history file stays script-friendly: timestamps are stored after commands as comments, so history lines can be copied into a script file.

Change the timestamp format:

```
bywaf> vars history.timestamp-format=%Y/%m/%d %H:%M:%S %Z
```

Scripts

Scripts are text files with one command expression per line:

```
# scan local host
hostscanner 127.0.0.1
portscanner --from-topic host.found
```

Blank lines and lines beginning with `#` are ignored. Inline comments are also allowed after whitespace:

```
plugins  # list loaded plugin providers
```

Run a script:

```
bywaf> load script=scan.bywaf
```

Bundled Commandlets

os

`ls` lists local files:

```
bywaf> ls
bywaf> ls bywaf/plugins
```

`cat` prints a local text file:

```
bywaf> cat README.md
```

`less` opens the system `less` pager when running interactively:

```
bywaf> less README.md
```

Use `/` to search inside `less`, arrow keys or paging keys to scroll, and `q` to quit.

discovery

`hostscanner` discovers live hosts with nmap:

```
bywaf> hostscanner 127.0.0.1
bywaf> hostscanner 192.168.0.1-255
bywaf> hostscanner 192.168.1-3.1-255
```

It emits `host.found` events.

network

`portscanner` scans hosts for open ports:

```
bywaf> portscanner 127.0.0.1
bywaf> portscanner --ports 22,80,443 127.0.0.1
```

If `--ports` is omitted, nmap uses its normal default top-port behavior. It emits `port.open` events.

Use listen mode to consume newly inserted hosts:

```
bywaf> portscanner --listen
```

http

`http_headers` performs HTTP HEAD requests and emits `http.headers` events:

```
bywaf> http_headers example.com
bywaf> http_headers --ssl true example.com
```

`http_probe` probes HTTP endpoints and emits `http.endpoint` events:

```
bywaf> http_probe https://example.com/
```

For authorized session-aware testing, it can use cookies:

```
bywaf> vars http_probe.cookie-file=/path/to/cookies.txt
bywaf> http_probe https://example.com/
bywaf> http_probe --firefox-profile ~/.mozilla/firefox/<profile>
```

Common Workflows

Scan one host for live status:

```
bywaf> hostscanner 127.0.0.1
```

Scan one host for ports:

```
bywaf> portscanner 127.0.0.1
```

Discover hosts and scan their ports:

```
bywaf> hostscanner 192.168.0.1-255 | portscanner
```

Run discovery and port scanning in the background:

```
bywaf> hostscanner 192.168.0.1-255 & | portscanner &
```

Probe HTTP services after port scanning:

```
bywaf> hostscanner 127.0.0.1 | portscanner --ports 80,443 | http_probe
```

Save the current database:

```
bywaf> save db=scan-results.sqlite3
bywaf> save --encrypt db=scan-results.sqlite3
```

Save variables and session history:

```
bywaf> save config=session.json
bywaf> save history=session.bywaf
```

Troubleshooting

If `python` is not available, use `python3`:

```
python3 -m bywaf
```

If scanning fails with an nmap error, verify that `nmap` is installed and that the selected Python nmap binding is available.

If socket creation is denied, the process may be running inside a sandbox or without the privileges needed by the selected scan type.

If a command is unknown, check loaded commandlets:

```
bywaf> cmds
```

If a plugin does not load, verify its directory structure and that it defines a `plugin()` factory returning a commandlet.

SQLite WAL files such as `.sqlite3-wal` and `.sqlite3-shm` are normal. Bywaf checkpoints the WAL on shutdown.

Developer Notes

The main package layout is:

```
bywaf/app.py          REPL and built-in commands
bywaf/runner.py       parsing, pipelines, foreground/background execution
bywaf/db.py           SQLite event store
bywaf/plugin.py       commandlet protocol and specs
bywaf/registry.py     plugin discovery and loading
bywaf/completion.py   prompt_toolkit/readline completion
bywaf/plugins/        bundled plugin providers
tests/                unit tests
```

Run tests:

```
python3 -m unittest discover -s tests
```

Add a commandlet by defining a class with a `CommandSpec` and a `run()` method, then expose it through a `plugin()` factory. Add bundled commandlets to `bywaf/plugins/plugins.json` when they should load by default.

Commandlets can declare completion metadata with `ArgumentSpec`, `OptionSpec(..., completion=CompletionSpec(...))`, or an optional custom `complete(context, args, prefix)` method. See `PLUGIN_AUTHOR_GUIDE.md` for a walkthrough and a small working example.

Reference

Useful built-ins:

```
help [command]
plugins
cmds
vars [name=value]
history
job <list|show|cancel|end|kill>
pipeline <list|show|cancel|end|kill|attach>
signal <job=id|run=id|serial=id> <action> [--soft|--hard] [key=value ...]
cancel <job=id|pipeline=id|run=id>
end [--soft|--hard] <job=id|pipeline=id|run=id>
kill [--soft|--hard] <job=id|pipeline=id|run=id>
name <run=id|pipeline=id|job=id> [name text]
note [add] <run=id|pipeline=id|job=id> [text=note|file=path]
artifact <attach|list|remove|replace|save|search|verify> [artifact=id|run=id|pipeline=id|job
search [--regexp] <name=text|note=text|content=text> [artifact=id|run=id|pipeline=id|job=id]
jobs
pipelines
runs
events [tail|--tail] [last=N]
topics
event <topic>
event job=<id>
event run=<id>
event pipeline=<id>
event serial=<id>
db <status|path|checkpoint|vacuum|new|encrypt|decrypt|rekey>
load plugin=<resource>
load script=<resource>
load db=<resource>
load config=<resource>
load history=<resource>
save db=<resource>
save --encrypt db=<resource>
```

```
save config=<resource>
save history=<resource>
prompt [pattern]
exit
```

Common event topics:

```
host.found
port.open
http.headers
http.endpoint
job.requested
job.claimed
job.started
job.finished
job.failed
```